

Video Game Programming: Fuzzy Logic & Genetic Algs

L14

Prof. Fabio Buttussi, PhD
Human-Computer Interaction Laboratory
Department of Mathematics, Computer Science, and Physics
University of Udine - Italy

Fuzzy logic

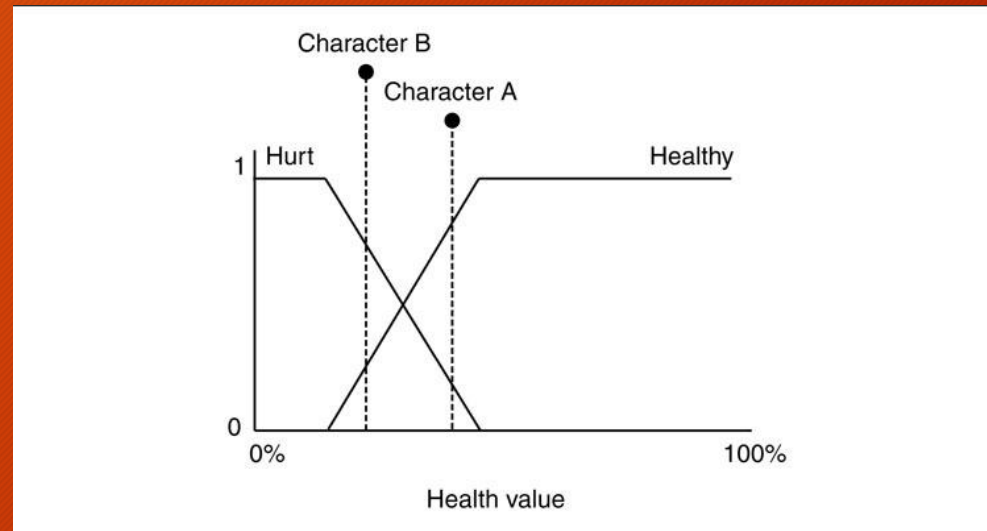
- Decision-making techniques seen in previous lessons ended up with “crisp” decisions (e.g., an NPC either does one action or another)
- **Fuzzy logic** is a set of mathematical techniques designed to cope with gray areas (e.g., an NPC is in a state to some extent, but also in others)
- Several academics **disregard** fuzzy logic in favor of probability-based approaches to represent any kind of uncertainty
- However, fuzzy logic has been used in several games and is applicable to many decision-making problems

Fuzzy set

- Traditional logic uses **predicates** to describe properties
- Each entity either has a predicate or not, i.e., it is either in or out the corresponding **classical set**
- Fuzzy logic extends the notion of a predicate by giving it a value
- Every entity can **partially** belong to the set, and some entities can belong to the set more than others
- These sets are called **fuzzy sets**, and the numeric value is called the **degree of membership**
- A common mistake is to interpret the value as a probability or a percentage

Membership of multiple sets

- In traditional logic, some predicates are mutually exclusive
- In fuzzy logic, an entity can belong to two or more mutually exclusive sets
- The fuzzy equivalent of mutual exclusion requires that membership degrees sum to 1

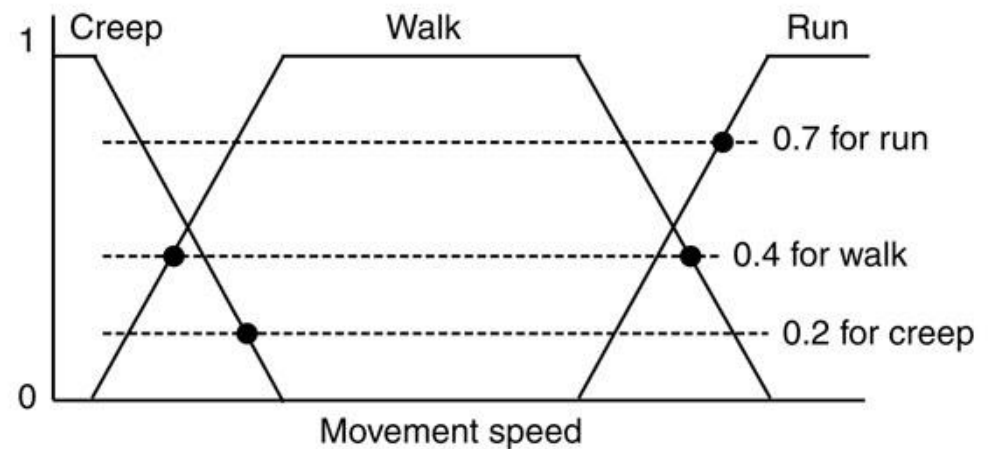


Fuzzification

- Turning data into degrees of membership is called **fuzzification**
- The most common technique is turning numeric values into memberships of fuzzy sets using a **membership function** (e.g., health points into healthy and hurt memberships)
- Games often use Boolean values and enumerations
- If a fuzzy set corresponds to a Boolean value, then the membership values will be 0 and 1
- For enumerated values, each possible value has a pre-determined stored membership value

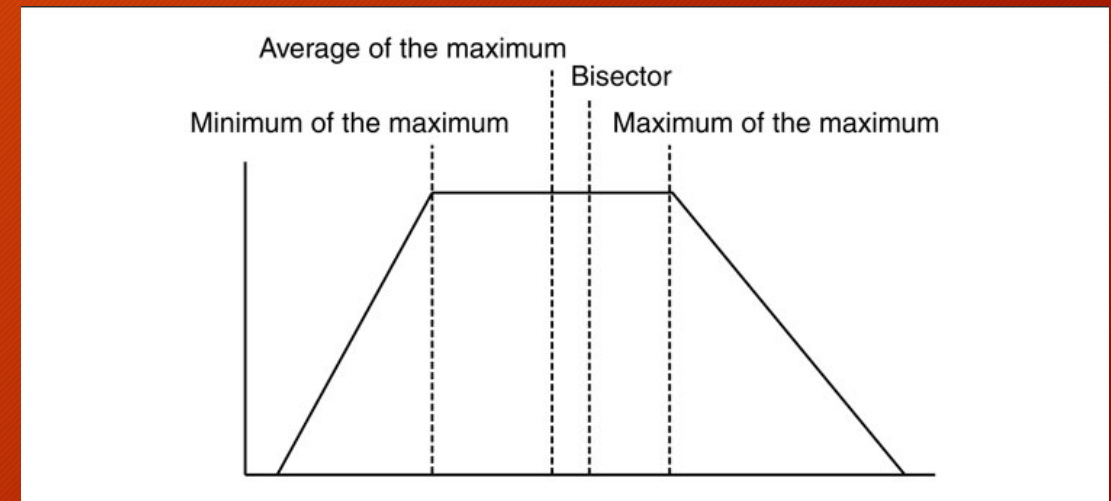
Defuzzification

- After applying a fuzzy logic, **defuzzification** should be used
- Defuzzification involves turning a set of membership values into a single output
- The output value is almost always a number
- Defuzzification is not simple and in several cases (as in fig.) there is not an obvious method



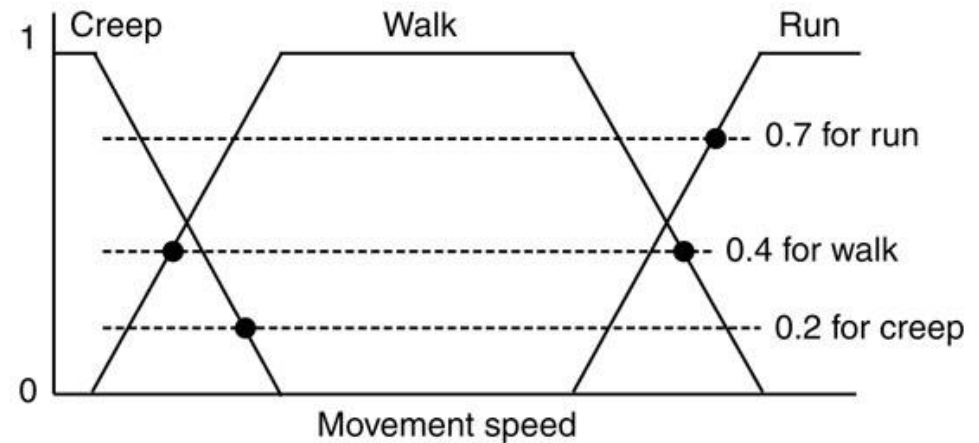
Highest membership

- One option is calculating the output choosing the fuzzy set with the greatest degree of membership
- There are four common points chosen:
 - minimum value at which function returns 1
 - maximum value
 - average of the two
 - bisector of the function (using pre-calculated integration)



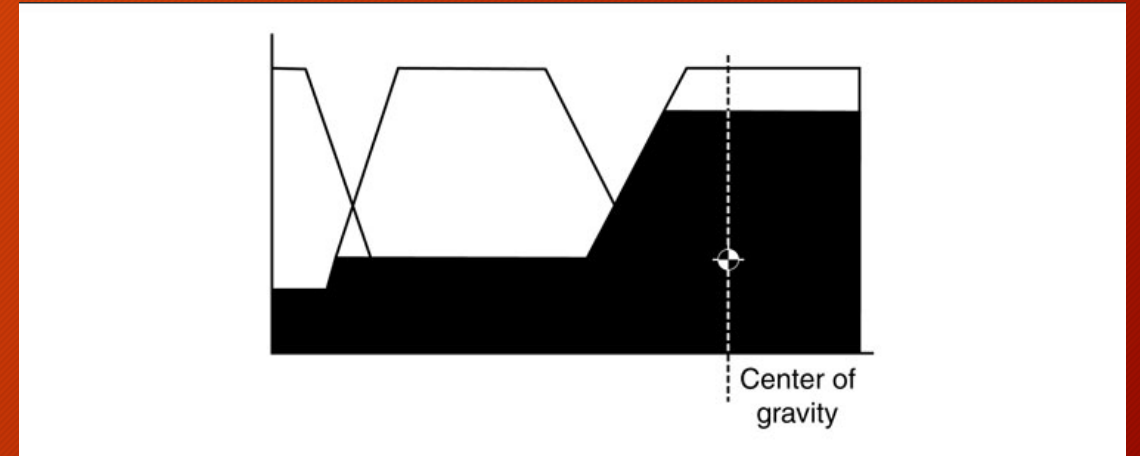
Blending based on membership

- Another option is blending each characteristic point based on its corresponding degree of membership
- In the example in fig., movement speed will be $(0.7 * v_{run} + 0.4 * v_{walk} + 0.2 * v_{creep}) / (0.7 + 0.4 + 0.2)$
- The v values as calculated with one of the four described methods



Center of gravity

- Another option is finding the center of gravity
- Each membership function is cropped at the membership value for its set
- The center of cropped regions is found by integration
- The point is used to calculate the output value
- Integration cannot be done offline because it varies with cropping level

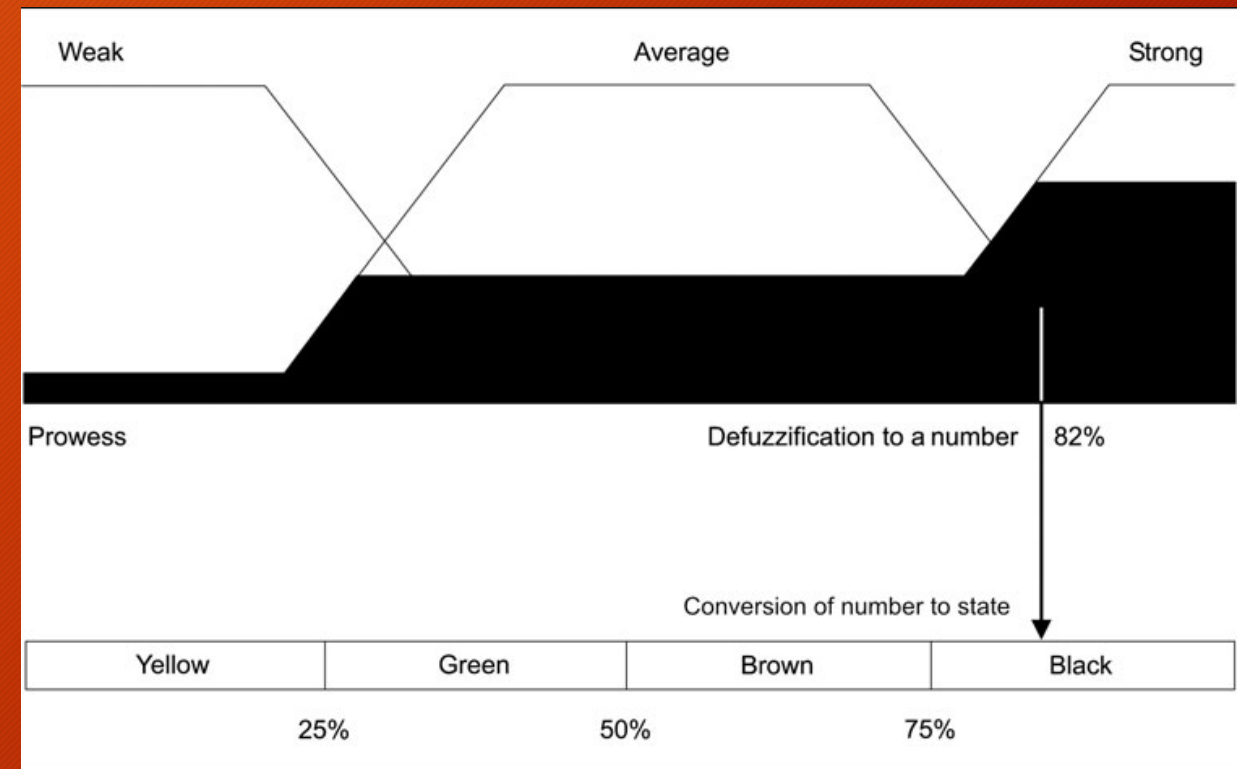


Choosing a defuzzification approach

- Names used in fuzzy logic are sometimes fuzzy
- E.g., IEEE version of center of gravity does not crop each function before calculating its center of gravity, so it is more like blending approaches described in previous slides
- Center of gravity is favored in many fuzzy logic applications
- Results provided by the blending approach is often good and quicker to calculate
- It could be further sped up by storing values for states (e.g., $v_{creep}=0.2\text{m/s}$) and then simply calculating blended averages

Defuzzification of Boolean and enum

- To produce a Boolean output, a cut-off value is used
- Enumerations that can be ordered are often defuzzified as a numerical value (in fig.)
- Enumerations that cannot be ordered are defuzzified by making sure a fuzzy set corresponds to each possible option and choosing the set / enum with the highest membership



Fuzzy logic operators

- As in traditional logic, operators (e.g., AND, OR, NOT) can be used to combine the truth of simple facts
- Since fuzzy logic use membership values (m in fig.), each operators corresponds to an equation
- Using 0 and 1 membership values, the results are the same of traditional logic

Expression	Equivalent	Fuzzy Equation
NOT A		$1 - m_A$
A AND B		$\min(m_A, m_B)$
A OR B		$\max(m_A, m_B)$
A XOR B	NOT(B) AND A OR NOT(A) AND B	$\max(\min(m_A, 1 - m_B), \min(1 - m_A, m_B))$
A NOR B	NOT(A OR B)	$1 - \max(m_A, m_B)$
A NAND B	NOT(A AND B)	$1 - \min(m_A, m_B)$

Fuzzy rules

- Fuzzy rules relate the known membership of certain fuzzy sets to generate new membership values for other fuzzy sets
- E.g., in a racing game, if we are close to the corner AND we are traveling fast, then we should brake: $m_{\text{brake}} = \min(m_{\text{corner}}, m_{\text{fast}})$
- Fuzzy rules can be used to determine if transitions in a state machine should fire
- They can be used also as rules of a rule-based system
- Some decision-making systems (fuzzy control) use only fuzzy rules

Fuzzy control

- In many problems a set of different actions can be carried out, but it is not always clear which one is best
- E.g., in a racing game, a car can brake at different intensities
- With a decision tree or state machine, the car will break or not
- Fuzzy control allows to handle intermediate values
- The input is a set of crisp inputs, each mapped into fuzzy states
- The output is a set of fuzzy states, representing possible actions
- Fuzzy rules link outputs to inputs

Fuzzy control rules

- Each clause in a rule is a state from a different crisp input
- Clauses are always combined with AND
- In fig., there are two clauses because there are two inputs, and each clause represents one state from each input
- It is a common requirement to have a complete set of rules (e.g., 2x2 in fig.)
- The final degree of membership for output states is the maximum output from any of the applicable rules

```
IF corner-entry AND going-fast THEN brake
IF corner-exit AND going-fast THEN accelerate
IF corner-entry AND going-slow THEN accelerate
IF corner-exit AND going-slow THEN accelerate
```

Corner-entry = 0.1

Corner-exit = 0.9

Going-fast = 0.4

Going-slow = 0.6

Brake = $\min(0.1, 0.4) = 0.1$

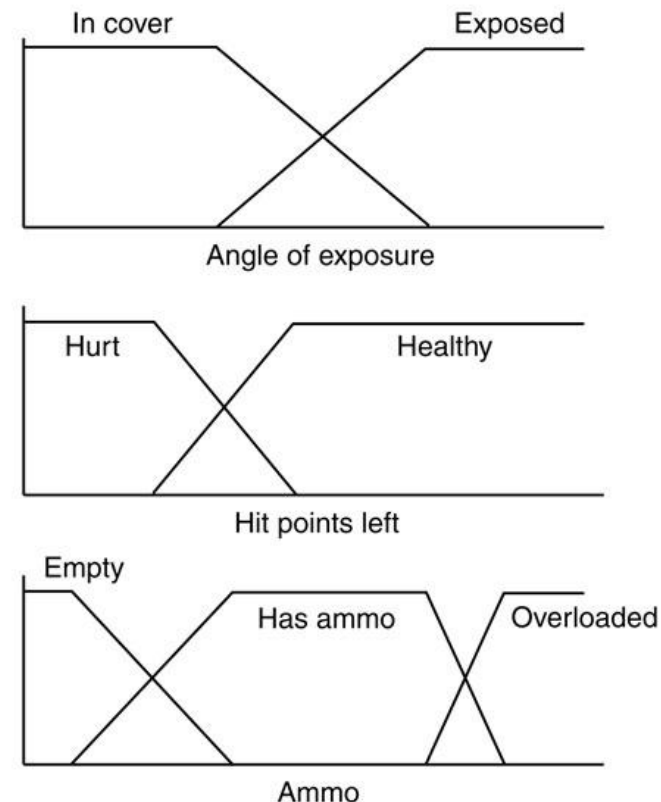
Accelerate = $\min(0.9, 0.4) = 0.4$

Accelerate = $\min(0.1, 0.6) = 0.1$

Accelerate = $\min(0.9, 0.6) = 0.6$

Fuzzy control scalability

- The main problem of this approach is scalability
- The example in fig. needs 12 rules ($2 \times 2 \times 3$)
- To process a system with 10 input variables, each with 5 states, would require almost 10 million rules
- This would be almost impossible to create



Fuzzy state machines

- A fuzzy state machine can be
 - a state machine where transitions use fuzzy logic to trigger
 - a state machine using fuzzy states rather than conventional states
 - a state machine with fuzzy states and transitions
- These slides will focus on the second, i.e., on state machines that instead having a single current state, can be in more states at the same time with some degree of membership (DOM) for each

Fuzzy state machine transitions

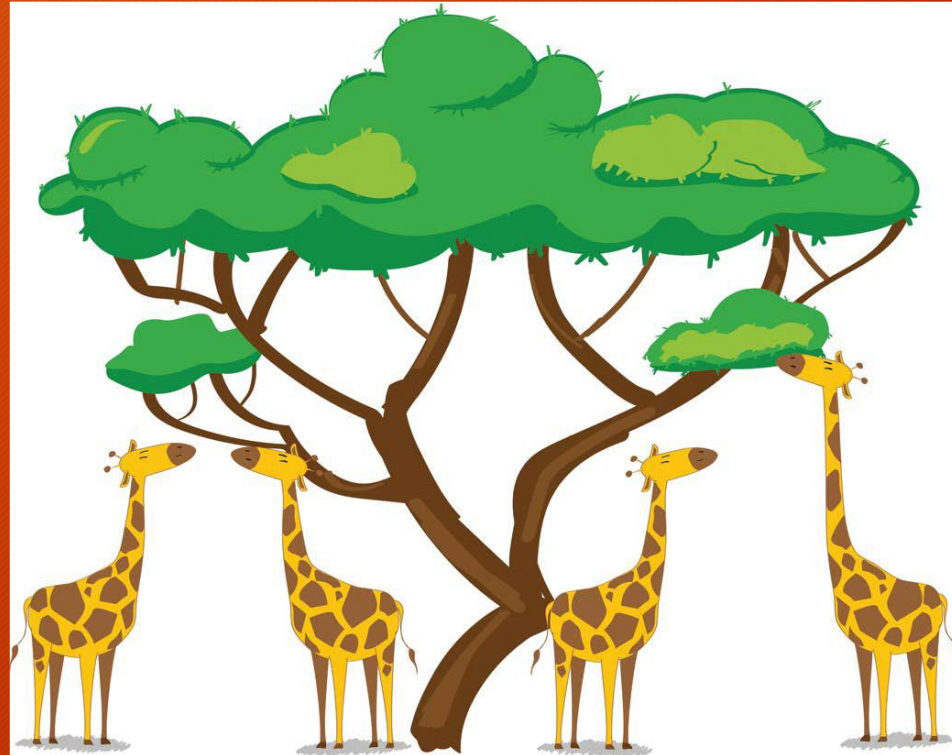
- Transitions belonging to all active states can trigger
- The first transition in each state is fired in decreasing DOM order
- If a transition fires, it can transition to any number of new states
- The transition itself also has an associated degree of transition (T)
- The DOM of the target state B is given by the DOM of the current state ANDed with T
- The new value of state B is also ORed with the existing value
- The start state of the transition is ANDed with NOT T

Transition example and application

- $MA = 0.4$, $MB = 0.3$, $MT = 0.6$
- $MB' = MB \text{ OR } (A \text{ AND } T) = \max(0.3, \min(0.4, 0.6)) = 0.4$
- $MA' = MA \text{ AND NOT } T = \min(0.4, 1 - 0.6) = 0.4$
- Transitions are triggered in the same way as for finite state machines, but tests can involve fuzzy logic
- The other modification needed is changing how actions are performed: an external defuzzification process will determine the action required considering all states (instead of doing something based on each single state)

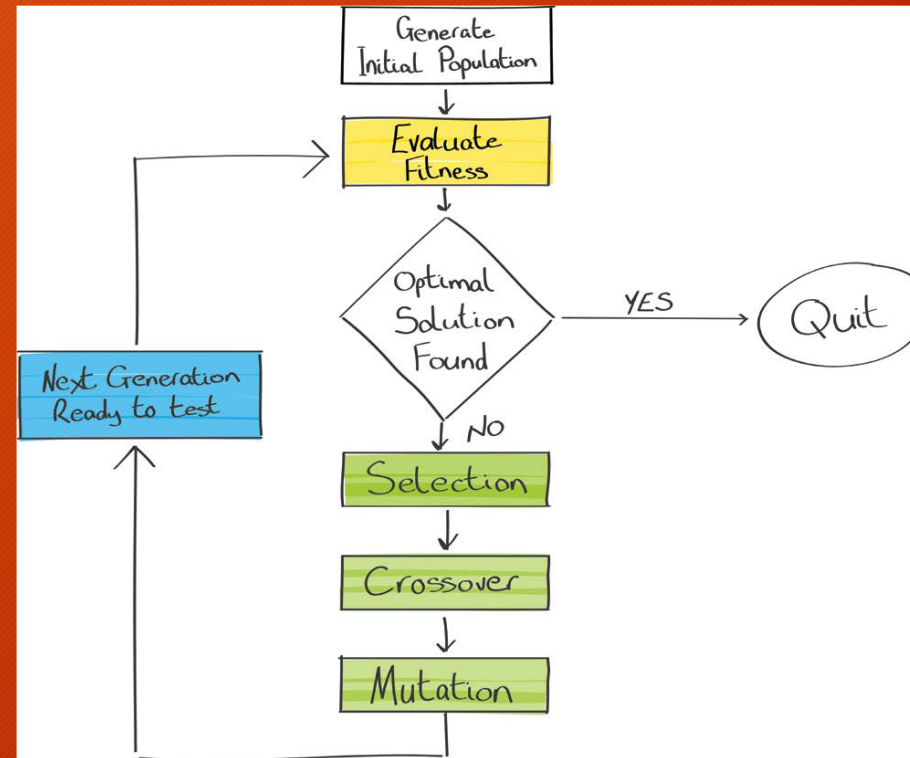
Genetic algorithms

- Genetic Algorithms are based upon Charles Darwin's principle of the survival of the fittest
- Multi-stepped approach: selection, reproduction, and mutation
- By combining solutions that are not good enough over multiple generations, we can evolve to a good solution



Overall functioning

- Starting from an initial population, fitness is evaluated
- Some individuals from the population are selected and combined
- Mutations are (rarely) applied
- Fitness of the new generation is tested...



Chromosomes

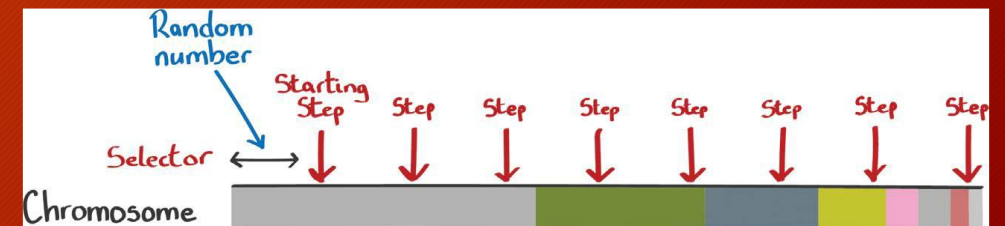
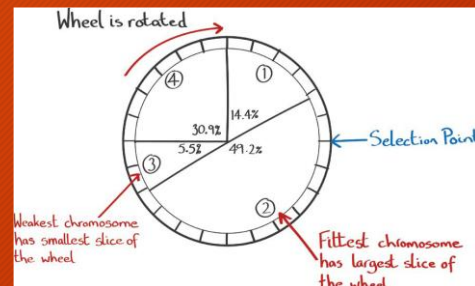
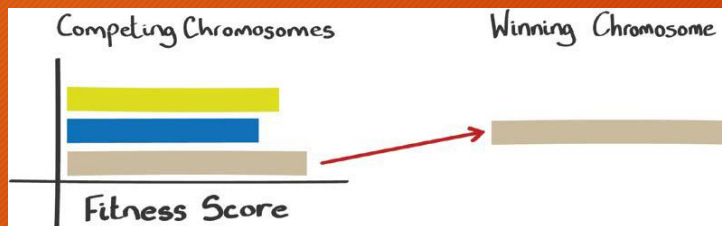
- We need a way to map individual characteristics to a chromosome
- An approach is having binary values, each indicating if the individual has or does not have a specific characteristic (see figs.)
- Other approaches use float values, enumerators, or even structs
- The initial population can be created using random values and/or based on previous knowledge

Index	Equipment	Ability
0	Gas mask	No damage from gas
1	Jetpack	Fast movement
2	Armour	Strength bonus
3	X-Ray goggles	See enemy locations
4	Grenades	Ranged attack
5	Whip	Melee attack
6	Pistol	Single distance shot
7	Gas canisters	Dropped. Give off noxious fumes

Utilities				Weapons			
0	0	1	0	1	1	0	0
index - 0	1	2	3	4	5	6	7

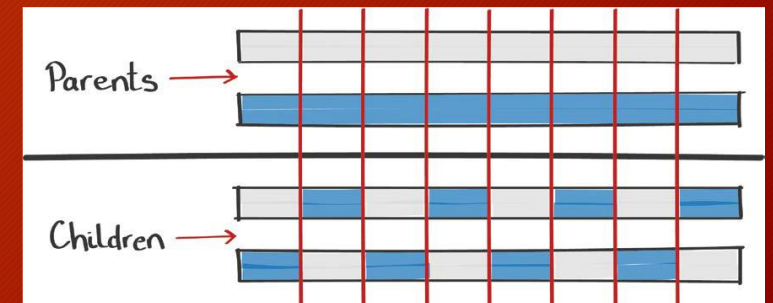
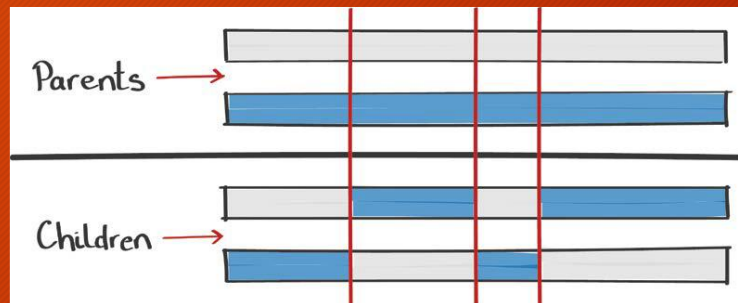
Selection

- Fitness is calculated by combining relevant values obtained by simulating the game with the current population
- From the current population, selection identifies the parents for the new population
- How many? Same number, but with multiple copies of the fitter
- How? Ranking, Tournament, Roulette Wheel, Stochastic Universal Sampling, Elitist...



Crossover

- Takes chromosomes from the parents' pool and combines them to create the child chromosomes
- Number of children as in the previous generation
- Suggest approach: two children from two parents using all
- How? 1-Point, n-Point, Uniform
- Rarely, a mutation can be applied (e.g., changing a single value)



Bibliography

- Roberts, Artificial Intelligence in Games, CRC Press, 2023
- Millington, AI for Games, 3rd ed., CRC Press, 2019

